Graph-Aware Transformers for Smart Contract Vulnerability Detection

Axel Bröns [1 2] **Valentin Kocijancic** [3 2] **Antoine Goudedranche** [1 2] **Thibault Garel** [1 2] **Hugo Rivière** [1 2]
Omar El Alami El Fellousse [4 2]

Abstract

Smart contracts manage billions of dollars in decentralized ecosystems, yet their immutable nature makes them highly susceptible to costly, unpatchable vulnerabilities. Current automated auditing tools rely on rigid, expert-defined static rules that fail to generalize, while modern generative Large Language Models (LLMs) struggle with deterministic logic and exhibit severe precision deficits. Furthermore, existing machine learning approaches either treat source code as flat text (ignoring structural execution logic) or employ pure Graph Neural Networks (GNNs) that discard crucial lexical semantics. In this paper, we propose a novel deep learning framework for Solidity vulnerability detection leveraging GraphCodeBERT, a Graph-Aware pre-trained Transformer. By integrating raw source code tokens with underlying Data Flow Graphs (DFGs) through a graph-guided masked attention mechanism, our approach explicitly learns anomalous data dependencies while preserving rich contextual semantics. Extensive experiments across multiple classification granularities demonstrate that our model significantly outperforms state-of-the-art zero-shot LLMs. In a strictly balanced binary classification setting, our fine-tuned architecture achieves an 87% F1-score and a critical 91% recall rate. To facilitate future research in AI-driven blockchain security, our datasets, preprocessing pipelines, and model weights are publicly available at https://github.com/axelbrons/gat-smart-contracts.

1. Introduction

Smart contracts are decentralized programs that execute automatically on the blockchain (Nakamoto & Bitcoin, 2008), removing the need for trusted intermediaries. Within this ecosystem, Ethereum (Buterin et al., 2014) stands out as the most prominent platform for decentralized applications, powered by its primary programming language, Solidity (Bauer, 2022). However, because blockchains are immutable, these contracts cannot be modified or patched with security updates once they are deployed. Consequently, simple coding errors can expose contracts to severe network attacks, as demonstrated by the infamous DAO hack that resulted in millions of dollars stolen (Sayeed et al., 2020). To date, such vulnerabilities have cost the blockchain ecosystem billions.

To mitigate these threats, the community primarily relies on automated vulnerability detection tools based on static and dynamic analysis. Unfortunately, these traditional methods depend on rigid, expert-defined rules. Because these rules cannot adapt to complex or novel attack patterns, they frequently suffer from high false-positive rates and scalability limits. While recent machine learning approaches like LSTMs (Sherstinsky, 2020) and Graph Neural Networks (GNNs) attempt to automate pattern recognition, they remain flawed: LSTMs ignore code structure, and pure GNNs often strip away crucial lexical details.

In this paper, we propose a novel deep learning approach to detect fraudulent Solidity code using GraphCodeBERT (Guo et al., 2021), a graph-aware pre-trained language model. Instead of relying on manual heuristics, our method characterizes a smart contract by capturing both its sequential source code tokens and its underlying Data Flow Graph (DFG). We formulate vulnerability detection as a binary classification problem, categorizing contracts as either safe or vulnerable, as illustrated in the overall pipeline of Figure 1. By utilizing a graph-guided attention mechanism, our fine-tuned model preserves the code's semantic meaning while explicitly learning critical data dependencies, significantly outperforming traditional static analysis tools.

2. Related Work

Traditional Security Analysis Tools Early vulnerability detection primarily relies on static and dynamic analysis. Major tools such as Oyente, Mythril, Securify, and SmartCheck (Sharma et al., 2022; Tsankov et al., 2018;

[1]Majeure Data & IA [2]ECE Ecole d'ingénieurs, 10 Rue Sextius Michel, 75015 Paris [3]Majeure Finance [4]Majeure Cybersécurité. Correspondence to: Axel Bröns <axel.brons@edu.ece.fr>.

Preprint. April 26, 2026.

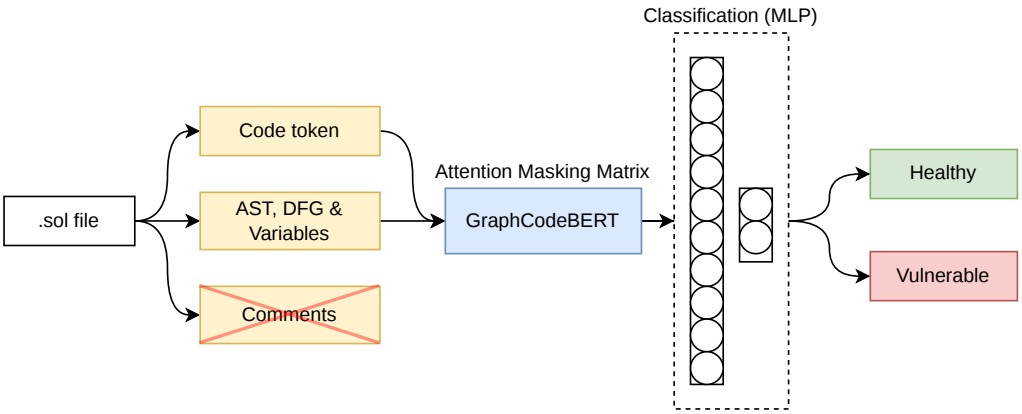

Figure 1. Overview of the proposed vulnerability detection pipeline. The raw Solidity (`.sol`) file is parsed to extract sequential code tokens alongside the Abstract Syntax Tree (AST) and Data Flow Graph (DFG) variables, while natural language comments are deliberately discarded to optimize context length. These features construct the input sequence and the graph-guided attention masking matrix for the GraphCodeBERT encoder. Finally, a Multi-Layer Perceptron (MLP) classification head processes the aggregated contextual embeddings to output a binary prediction (Healthy or Vulnerable).

Tikhomirov et al., 2018) have been widely adopted to identify common bugs like Reentrancy and Integer Overflows. However, research indicates that these tools provide a "dangerously false sense of security". For instance, experiments show that Oyente fails to log 72.9% of Transaction Ordering Dependence (TOD) vulnerabilities and only detects 20.2% of known Parity Wallet hacks. Furthermore, tools like Securify are unable to recognize numerical overflows because they do not utilize numerical analysis. These limitations stem from a heavy reliance on rigid, expert-defined rules that cannot scale to detect novel or complex attack patterns.

NLP and Large Language Models (LLMs) To overcome the rigidity of static rules, researchers have recently explored Large Language Models (LLMs) like ChatGPT for vulnerability detection (Chen et al., 2025). While LLMs demonstrate a high recall rate, reaching up to 88.2% for GPT-4, their precision remains insufficient for practical security applications, often hovering around 22.6%. LLMs suffer from "Protected Mechanism Bias," where they overemphasize the mere presence of security modifiers (e.g., onlyOwner) without understanding the underlying logic. Moreover, their lack of an execution environment and specialized domain knowledge leads to significant false positives and an inability to handle code exceeding strict token limits.

Machine Learning and Graph-Based Approaches To move beyond manual heuristics, recent studies have explored machine learning. While sequential models like LSTMs treat code as flat text, ignoring the structural logic, Graph Neural Networks (GNNs) attempt to capture the contract's topology (Liu et al., 2021). However, the conversion of source code into a graph often discards crucial lexical and syntactic details. Furthermore, state-of-the-art frameworks like Vandal (Brent et al., 2018) or Zeus (Kalra et al., 2018) face engineering hurdles; for example, Vandal's decompiler often crashes when transforming EVM stack-based operations into register-based representations , and Zeus cannot fully validate mathematical equations.

Positioning of Our Study Our approach bridges the gap between lexical semantics and structural topology by fine-tuning GraphCodeBERT (Guo et al., 2021) on Solidity code. Unlike LLMs that lack precision or graph models that ignore natural language context (such as comments), our method leverages a graph-guided masked attention mechanism. By combining raw code tokens with the Data Flow Graph (DFG), we preserve both the rich semantic meaning of the source code and its functional dependencies. This synergy eliminates the reliance on manual rules while capturing the deep contextual logic required to identify sophisticated vulnerabilities that current tools fail to detect.

3. Data Preprocessing

The quality and representativeness of the training data are paramount for the generalization capabilities of deep learning models. To robustly train and evaluate our GraphCodeBERT-based architecture, we assembled and curated a comprehensive dataset of both vulnerable and verified safe smart contracts.

Data Sourcing Our dataset originates from two primary sources to ensure a wide diversity of coding patterns and security flaws:

- **Vulnerable Contracts:** The malicious and flawed Solidity codes were obtained from a publicly available dataset from (Liu et al., 2023) where we took files

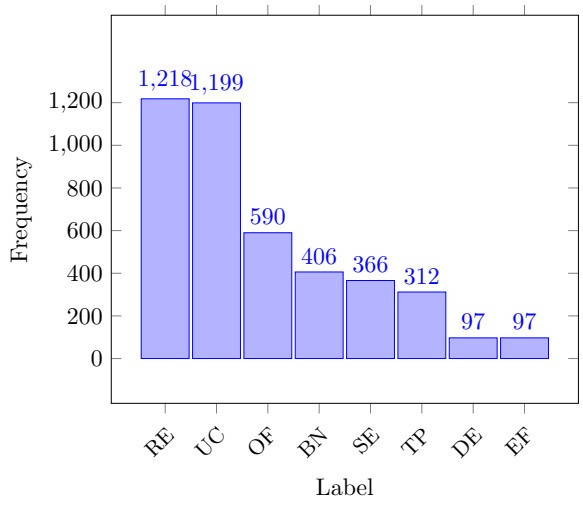

Figure 2. Repartition of the 8 label in the original dataset, where RE is reentrancy, UC is unchecked external call, Of is integer overflow, BN is block number dependency, SE is ether strict equality, TP is timestamp dependency and EF is ether frozen

from this link Kaggle link from TranDuongMinhDai. This repository contains a diverse taxonomy of vulnerabilities, explicitly categorized into eight distinct attack vectors: Reentrancy, Unchecked External Call, Overflow, Number Dependency, Strict Equality, Timestamp Dependency, Dangerous Delegatecall, and Ether Frozen. The initial distribution of these vulnerability classes is illustrated in Figure 2, highlighting the natural frequency disparities among different types of exploits.

- **Safe Contracts:** To represent healthy, production-ready code, we collected verified smart contracts from a widely recognized GitHub repository from etherscan_verified_contracts github link. This source provided a massive pool of approximately 46,000 verified and secure Solidity source codes.

Experimental Configurations To comprehensively evaluate the model's performance, from fine-grained vulnerability categorization to robust generalized detection, we engineered three distinct dataset configurations:

- **9-Class Setup:** The baseline dataset incorporated all 8 vulnerability types alongside the Safe class. This configuration posed a highly complex 9-label classification problem. The dataset distribution, detailed in Table 1, was balanced by limiting each category to 600 samples, with the exception of 'safe' data, which was set at 1000.
- **7-Class Setup:** Upon statistical analysis of the dataset, we observed that the two least represented vulnerability classes lacked sufficient data points for effective optimization via our method. To prevent statistical noise and improve the model's convergence, we discarded these two minority classes, formulating a more robust and representative 7-label dataset, we can see the detailed repartition in Table 1.
- **Binary Setup (Strictly Balanced):** For the primary evaluation of our model's core detection capability, we aggregated all vulnerability types into a single Vulnerable label. To ensure mathematical parity and completely eliminate baseline bias, we downsampled the data to create a perfectly balanced dataset (50% / 50%). This optimal configuration consists of exactly 4,000 vulnerable contracts and 4,000 safe contracts, randomly sampled using a fixed seed for reproducibility. The exact distribution and breakdown of the samples for this final configuration are detailed in Table 1.

Table 1. Table representing the number of smart contracts in each dataset created

Label	RE	UC	OF	BN	SE	TP	DE	EF	Safe
Original	1218	1199	590	406	366	312	97	97	–
9-Class	600	600	590	406	366	312	97	97	1000
7-Class	600	600	590	406	366	312	–	–	1000
Binary	4000								4000

4. Proposed Methodology

4.1. Graph-Aware Transformer Encoder

Figure 1 presents the high-level architecture of our proposed framework. Our approach leverages GraphCodeBERT (Guo et al., 2021), a pre-trained bidirectional language model that integrates source code sequences with their underlying semantic structures. Consider a smart contract represented by its code sequence $C = \{c_1, c_2, \ldots, c_n\}$ and its data flow variables $V = \{v_1, v_2, \ldots, v_m\}$ extracted from the Abstract Syntax Tree (AST) (Zhang et al., 2019). The model input X is constructed by concatenating these sequences with special tokens:

$$X = [[\text{CLS}], c_1, \ldots, c_n, [\text{SEP}], v_1, \ldots, v_m, [\text{SEP}]]$$

Note that while the original GraphCodeBERT architecture incorporates natural language comments (W) into its input sequence (Guo et al., 2021), we deliberately omit them in our formulation. In the context of smart contract security, vulnerabilities strictly reside within the execution logic and data dependencies rather than human-readable text. Stripping comments prevents the model from learning spurious

lexical correlations and frees up the 512-token context window (L) to maximize the retention of critical source code (C) and data flow variables (V).

To incorporate structural dependencies without altering the Transformer architecture, we inject data flow relations through a 3D attention masking matrix $M \in \{0,1\}^{L\times L}$, where L is the maximum sequence length (set to 512). Within each multi-head attention layer, the computation is constrained by this topology. Given the queries Q, keys K, and values V, the masked attention is defined as:

$$\text{Attention}(Q,K,V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}} + \widetilde{M}\right)V$$

where d_k is the scaling factor and $\widetilde{M}$ is the penalty matrix derived from the graph:

$$\widetilde{M}_{i,j} = \begin{cases} 0 & \text{if } M_{i,j} = 1 \\ -\infty & \text{if } M_{i,j} = 0 \end{cases}$$

This mechanism ensures that information flows exclusively between relevant textual tokens and semantically linked variables, effectively filtering the syntax noise inherent in Solidity code.

4.2. Classification Head and Training Objective

Following the deep encoder (comprising 12 Transformer layers), we extract the final state of the aggregation token, $h_{[\text{CLS}]} \in \mathbb{R}^{d_{model}}$, where $d_{model} = 768$. To prevent overfitting, we apply a dropout layer with probability $p = 0.1$. The resulting vector is then projected into the target class space ($C = 2$ for binary classification) via a linear transformation:

$$Z = W \cdot \text{Dropout}(h_{[\text{CLS}]}) + b$$

where Z denotes the logit vector, $W \in \mathbb{R}^{C\times d_{model}}$ is the weight matrix, and $b \in \mathbb{R}^C$ is the bias vector. The model is optimized by minimizing the Cross-Entropy loss. For a batch of size N, let $y_{i,c}$ be the one-hot encoded ground truth and $Z_{i,c}$ the logit for class c of the i-th sample. The objective function is defined as:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log\left(\frac{\exp(Z_{i,c})}{\sum_{j=1}^{C}\exp(Z_{i,j})}\right)$$

4.3. Discriminative Fine-Tuning and AdamW Optimization

Model parameters are updated non-uniformly to balance knowledge preservation and task adaptation. We employ the AdamW optimizer with discriminative learning rates: the pre-trained encoder is updated with a conservative rate ($\eta_{enc} = 10^{-5}$) to preserve latent code representations, while the classification head is trained more aggressively ($\eta_{cls} = 10^{-4}$). To further regularize the model and prevent weight explosion, an L_2 weight decay of $\lambda = 0.01$ is applied across all layers.

4.4. Cosine Annealing with Warmup

To prevent premature convergence and stabilize the pre-trained weights during initial updates (Goyal et al., 2017; Vaswani et al., 2017), we dynamically modulate the learning rate η_t using a Cosine Annealing schedule with warmup (Loshchilov & Hutter, 2016). During the initial 10% of the total iterations (t_{warmup}), η_t increases linearly to reach its peak value. Subsequently, it follows a cosine decay schedule :

$$\eta_t = \eta_{max} \times \frac{1}{2}\left(1 + \cos\left(\pi\frac{t - t_{warmup}}{T - t_{warmup}}\right)\right)$$

where t denotes the current training step and T represents the total number of steps. This scheduling strategy promotes stable feature extraction during the initial phase and a more refined convergence in the later stages.

4.5. Implementation Details and Training Efficiency

All experiments were conducted using the PyTorch framework. To mitigate hardware constraints and optimize the training pipeline, we implemented several efficiency techniques. First, to alleviate CPU bottlenecks caused by the intensive Abstract Syntax Tree (AST) parsing via Tree-sitter, we employed asynchronous data loading (`num_workers=4`, `pin_memory=True`), ensuring uninterrupted GPU utilization. Furthermore, to maximize VRAM efficiency and accelerate backpropagation, the network leverages Automatic Mixed Precision (AMP). Forward passes and matrix multiplications are projected into 16-bit floating-point (FP16), while weight updates are maintained in FP32 to preserve precision. Gradient underflow is dynamically prevented using a `GradScaler` prior to differentiation.

5. Results and Discussion

To evaluate the effectiveness of our proposed Graph-Aware Transformer architecture, we conducted a two-phase experimental study. First, we established a baseline using state-of-the-art Large Language Models (LLMs) via zero-shot prompting. Second, we evaluated our fine-tuned GraphCodeBERT approach across multiple classification granularities (9-class, 7-class, and binary) to assess its discriminative capabilities.

5.1. Baseline Evaluation: The Limitations of LLMs

Given the recent success of generative AI in software engineering, our initial experiments investigated whether off-the-shelf, instruction-tuned LLMs could accurately detect smart

contract vulnerabilities through zero-shot prompting. We evaluated several prominent models, including `llama3.2`, `deepseek-coder:6.7b`, and `qwen2.5-coder:7b`, measuring both precision and inference time over a subset of our dataset.

As shown in Table 2, the performance of generative LLMs was highly unsatisfactory for robust security auditing. The best-performing model (`llama3.2`) achieved a maximum precision of only 34.00%, while code-specific models like `deepseek-coder:6.7b` and `qwen2.5-coder:7b` completely failed to generalize, yielding precisions below 7%. These poor results can be attributed to the inherent nature of generative models: they excel at next-token prediction but struggle with deterministic, logic-based classification tasks without specialized fine-tuning. Furthermore, LLMs suffer from a significant computational overhead. The inference time ranged from 9 to over 47 minutes for a relatively small batch of contracts, rendering them impractical for large-scale, real-time blockchain security scanning. This failure highlights the necessity of our dedicated, embedding-based approach. Concerning the fine-tuned model TinyLlama-1.1B with LoRA (Hu et al., 2021), constrained by the prompt design, which enforces a 9-class classification task (labels 0 to 8) and requires the LLM to output exclusively a corresponding digit, the best fine-tuned model reached syntactically 99.21% accuracy (4,036/4,068), yielding only 32 invalid predictions.

Table 2. Baseline evaluation of various LLMs using zero-shot prompting for vulnerability detection. Generative models exhibit severe precision deficits and high inference latency. In blue we can see our models fine-tuned by LoRA

Models	Time *(min:sec)*	Precision (%)
`llama3.2`	22:48	**34.00**
`llama3.2`	17:33	29.47
`llama3.2`	**09:03**	24.00
`TinyLlama-1.1B LoRa`	–	22.20
`deepseek-coder:6.7b`	43:57	6.25
`kimi-k2.5:cloud`	47:14	6.00
`qwen2.5-coder:7b`	–	4.00
`codellama`	32:19	2.13
`mistral`	26:16	2.00

5.2. Performance of the Proposed Method

Moving beyond standard LLMs, we evaluated our fine-tuned GraphCodeBERT model. The global performance metrics across our three dataset configurations are summarized in Table 3.

Multi-Class Categorization Challenges In the 9-class configuration, the model achieved an accuracy of 0.60. While significantly better than random guessing, a detailed analysis reveals extreme inter-class disparities. As illustrated in Figure 3, the model successfully learned features for classes like Integer Overflow (OF, F1=0.77) and Block Number Dependency (BN, F1=0.71). However, it completely failed to identify Dangerous Delegatecall (DE, F1=0.00) and struggled with Ether Frozen (EF, F1=0.44). This failure is primarily due to severe data starvation (only 97 samples available in the original dataset for these classes) and the high semantic similarity these vulnerabilities share with standard, safe functional logic. By trimming the dataset to a 7-class setup (removing DE and EF), overall accuracy improved slightly to 0.63 (see Appendix 6), but inter-class confusion between structurally similar vulnerabilities (e.g., Unchecked Call vs. Reentrancy) remained a limiting factor.

Table 3. Global performance of the fine-tuned GraphCodeBERT model across different classification complexities.

Dataset Setup	Accuracy	Macro F1
9-Class	0.60	0.54
7-Class	0.63	0.63
Binary	**0.87**	**0.87**

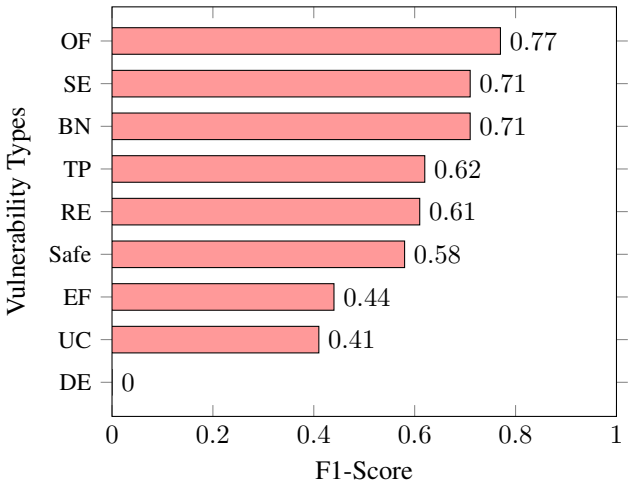

Figure 3. F1-Scores per category (9-class model). The model utterly fails to distinguish the DE vulnerability (0.00) and struggles with UC and EF, illustrating the challenge of extreme multi-class imbalance.

Binary Classification: Achieving High Efficacy The core objective of a security auditor is first to flag whether a contract is malicious or safe before diving into the specific taxonomy. In our strictly balanced Binary Setup, our model achieved a remarkable **87% accuracy** and an **87% F1-Score** (Table 4). By forcing the model to focus purely on the dichotomy between safe and vulnerable data flow patterns, the Graph-Aware Transformer effectively filtered out the noise that plagued the multi-class setup.

Crucially, in cybersecurity, False Negatives (predicting a malicious contract as safe) are catastrophically more expensive than False Positives (predicting a safe contract as malicious).

Figure 4 demonstrates that our model aligns perfectly with this security requirement. Out of 600 vulnerable contracts in the test set, the model correctly identified 544 (a recall of 91%, as shown in Table 4), misclassifying only 56 as safe.

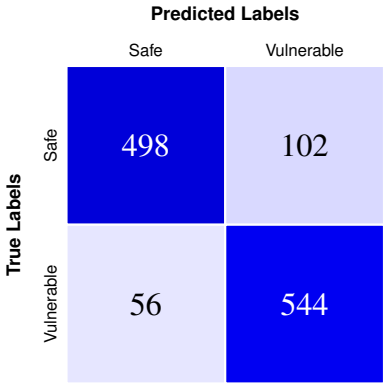

Figure 4. Confusion Matrix for binary classification (Safe vs. Vulnerable). The model demonstrates exceptional capability in identifying vulnerable codes (544 True Positives, representing a 91% recall rate).

Table 4. Detailed classification report for the binary model.

Class	Precision	Recall	F1-Score	Support
Safe	**0.90**	0.83	0.86	600
Vulnerable	0.84	**0.91**	**0.87**	600
Accuracy			0.87	1200
Macro Avg	0.87	0.87	0.87	1200
Weighted Avg	0.87	0.87	0.87	1200

5.3. Discussion

The stark contrast in performance between generative LLMs and our GraphCodeBERT approach emphasizes the importance of structural representation in code analysis. While LLMs attempt to infer vulnerabilities purely from textual patterns, often falling victim to superficial syntax or "Protected Mechanism Bias", our method explicitly routes information through the Data Flow Graph (DFG).

By masking the Transformer's self-attention to align with actual data dependencies, our model learns the true underlying mechanics of the smart contract (e.g., how money flows or how states are updated) rather than just memorizing vocabulary. This explains the exceptionally high recall (91%) in the binary classification task: the model is highly sensitive to anomalous data flows that typically characterize exploits like reentrancy and integer overflows.

6. Limitations

While our Graph-Aware Transformer demonstrates significant improvements over traditional static analysis and standard LLMs, this study presents several practical and theoretical limitations that must be acknowledged.

Computational Constraints and Fine-Tuning Duration Fine-tuning a deep, 12-layer Transformer architecture like GraphCodeBERT is highly resource-intensive. Due to hardware and budgetary constraints, our training process was restricted in terms of total epochs and extensive hyperparameter sweeping. Consequently, the model may not have reached its absolute optimal convergence. We hypothesize that extended training durations, coupled with larger batch sizes distributed across multiple GPUs, could yield further performance gains and stabilize the learning curve.

Dataset Scale and Scarcity Although we implemented a rigorous 50/50 balancing strategy to mitigate class imbalance, the absolute size of our annotated dataset remains relatively small by deep learning standards (8,000 total contracts in the binary setup). The scarcity of high-quality, manually verified datasets for smart contract vulnerabilities restricts the model's exposure to a wider variety of coding styles, complex inheritance structures, and architectural edge cases, which may impact its real-world generalization.

Solidity Version Discrepancy A critical limitation of our study stems from the rapid evolution of the Solidity programming language. The vulnerable contracts sourced from our Kaggle dataset are predominantly written in older compiler versions (specifically Solidity v0.4.x). However, the Ethereum ecosystem has since migrated to modern versions (v0.8.x and above), which introduce fundamental changes at the compiler level, such as built-in arithmetic overflow and underflow protections. Because our model's weights were optimized on legacy syntax and older vulnerability patterns, its predictive accuracy may decrease when analyzing state-of-the-art contracts that utilize modern paradigms.

Detection of Zero-Day Vulnerabilities Finally, our methodology inherently relies on a supervised learning paradigm. The network is explicitly optimized to recognize the structural and data-flow patterns of the specific vulnerability classes present in the training data (e.g., Reentrancy, Unchecked Calls). As a result, the system is fundamentally limited in its ability to detect "zero-day" exploits, entirely novel attack vectors or newly discovered logical flaws that exhibit topological signatures unseen during the training phase.

7. Conclusion

In this study, we addressed the critical challenge of automated smart contract vulnerability detection, a domain where traditional static analysis tools and modern generative Large Language Models (LLMs) frequently fall short. By

demonstrating the severe precision deficits and high inference latencies of zero-shot LLMs, we highlighted the necessity of moving beyond flat, text-based code analysis. To this end, we proposed a novel deep learning framework that fine-tunes GraphCodeBERT, a Graph-Aware Transformer, to synergize the rich lexical semantics of Solidity source code with the topological rigor of Data Flow Graphs (DFGs).

Our experimental results validate the efficacy of this hybrid approach. While extreme class imbalance and data starvation pose ongoing challenges for fine-grained multi-class categorization, our model excels at the primary objective of security auditing: binary threat detection. By successfully identifying malicious contracts with an 87% F1-score and a 91% recall rate, our graph-guided attention mechanism proves highly adept at filtering out syntactic noise and capturing the anomalous data dependencies indicative of exploits. Ultimately, this research establishes that integrating structural code logic into pre-trained language models offers a robust, scalable paradigm for securing decentralized financial ecosystems.

7.1. Future Work

Building upon the foundational findings of this study, several promising avenues for future research emerge to address current limitations and elevate the model's real-world applicability:

Dataset Modernization and Expansion: Future iterations must prioritize the collection and annotation of smart contracts written in modern Solidity compiler versions (v0.8.x and above). Expanding the dataset to include multi-contract protocols and complex inheritance structures will ensure the model's relevance against contemporary development paradigms.

Extended Context Architectures: To overcome the strict 512-token limit of the current GraphCodeBERT architecture, future research should explore integrating sparse attention mechanisms (such as those used in Longformer) or hierarchical chunking strategies. This would allow the network to process massive, enterprise-grade smart contracts in their entirety without truncating critical execution logic.

Unsupervised Anomaly Detection for Zero-Day Exploits: Because supervised learning is inherently restricted to known vulnerability taxonomies, transitioning toward semi-supervised or unsupervised learning is a critical next step. By training deep autoencoders or contrastive learning models exclusively on verified safe contracts, future systems could flag deviations and detect novel "zero-day" exploits based on topological anomalies rather than predefined attack signatures.

References

Bauer, D. P. *Solidity*, pp. 13–16. Apress, Berkeley, CA, 2022. ISBN 978-1-4842-8045-4. doi: 10.1007/978-1-4842-8045-4_2. URL https://doi.org/10.1007/978-1-4842-8045-4_2.

Brent, L., Jurisevic, A., Kong, M., Liu, E., Gauthier, F., Gramoli, V., Holz, R., and Scholz, B. Vandal: A scalable security analysis framework for smart contracts. *arXiv preprint arXiv:1809.03981*, 2018.

Buterin, V. et al. A next-generation smart contract and decentralized application platform. *white paper*, 3(37): 2–1, 2014.

Chen, C., Su, J., Chen, J., Wang, Y., Bi, T., Yu, J., Wang, Y., Lin, X., Chen, T., and Zheng, Z. When chatgpt meets smart contract vulnerability detection: How far are we? *ACM Transactions on Software Engineering and Methodology*, 34(4):1–30, 2025.

Goyal, P., Dollár, P., Girshick, R., Noordhuis, P., Wesolowski, L., Kyrola, A., Tulloch, A., Jia, Y., and He, K. Accurate, large minibatch sgd: Training imagenet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S. K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., and Zhou, M. Graphcodebert: Pre-training code representations with data flow, 2021. URL https://arxiv.org/abs/2009.08366.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. Lora: Low-rank adaptation of large language models, 2021. URL https://arxiv.org/abs/2106.09685.

Kalra, S., Goel, S., Dhawan, M., and Sharma, S. Zeus: analyzing safety of smart contracts. In *Ndss*, pp. 1–12, 2018.

Liu, Z., Qian, P., Wang, X., Zhuang, Y., Qiu, L., and Wang, X. Combining graph neural networks with expert knowledge for smart contract vulnerability detection. *IEEE Transactions on Knowledge and Data Engineering*, 35 (2):1296–1310, 2021.

Liu, Z., Qian, P., Yang, J., Liu, L., Xu, X., He, Q., and Zhang, X. Rethinking smart contract fuzzing: Fuzzing with invocation ordering and important branch revisiting. *arXiv preprint arXiv:2301.03943*, 2023.

Loshchilov, I. and Hutter, F. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983*, 2016.

Nakamoto, S. and Bitcoin, A. A peer-to-peer electronic cash system. *Bitcoin.–URL: https://bitcoin. org/bitcoin. pdf*, 4(2):15, 2008.

Sayeed, S., Marco-Gisbert, H., and Caira, T. Smart contract: Attacks and protections. *Ieee Access*, 8:24416–24427, 2020.

Sharma, N., Sharma, S., et al. A survey of mythril, a smart contract security analysis tool for evm bytecode. *Indian Journal of Natural Sciences*, 13(75):51003–51010, 2022.

Sherstinsky, A. Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network. *Physica d: Nonlinear phenomena*, 404:132306, 2020.

Tikhomirov, S., Voskresenskaya, E., Ivanitskiy, I., Takhaviev, R., Marchenko, E., and Alexandrov, Y. Smartcheck: Static analysis of ethereum smart contracts. In *Proceedings of the 1st international workshop on emerging trends in software engineering for blockchain*, pp. 9–16, 2018.

Tsankov, P., Dan, A., Drachsler-Cohen, D., Gervais, A., Buenzli, F., and Vechev, M. Securify: Practical security analysis of smart contracts. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, pp. 67–82, 2018.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

Zhang, J., Wang, X., Zhang, H., Sun, H., Wang, K., and Liu, X. A novel neural source code representation based on abstract syntax tree. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pp. 783–794. IEEE, 2019.

A. Detailed report of our tests

Table 5. Detailed classification report for the 9-class model.

Class	Precision	Recall	F1-Score	Support
BN	0.69	0.73	0.71	30
DE	0.00	0.00	0.00	15
EF	0.35	0.60	0.44	15
SE	0.77	0.67	0.71	30
OF	0.71	0.83	0.77	30
RE	0.54	0.70	0.61	30
TP	0.59	0.67	0.62	30
UC	0.64	0.30	0.41	30
Safe	0.59	0.57	0.58	30
Accuracy			0.60	240
Macro Avg	0.54	0.56	0.54	240
Weighted Avg	0.59	0.60	0.58	240

Table 6. Detailed classification report for the 7-class model.

Class	Precision	Recall	F1-Score	Support
BN	0.76	0.84	0.80	61
SE	0.71	0.55	0.62	55
OF	0.84	0.89	0.86	89
RE	0.43	0.61	0.51	90
TP	0.72	0.60	0.65	47
UC	0.42	0.24	0.31	90
Safe	0.65	0.69	0.67	150
Accuracy			0.63	582
Macro Avg	0.65	0.63	0.63	582
Weighted Avg	0.63	0.63	0.63	582

Table 7. Detailed classification report for the binary model.

Class	Precision	Recall	F1-Score	Support
Safe	0.90	0.83	0.86	600
Vulnerable	0.84	0.91	0.87	600
Accuracy			0.87	1200
Macro Avg	0.87	0.87	0.87	1200
Weighted Avg	0.87	0.87	0.87	1200